



COMPROMETIDOS CON LA
ACREDITACIÓN

¡Algo bueno está pasando!

#PonteEnModoAcreditación

Manual Técnico Aplicación - API de Pronóstico Hidrológico Integrado (IDEAM)

Integrantes:

Ing. Julian Andres Quimbayo Castro

Ing. José Miguel Llanos Mosquera

Est. Karen Johana Caicedo Arias

Est. Oscar Guillermo Sierra Lozano

Facultad de Ingeniería

Ingeniería de Sistemas





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Introducción

El presente manual técnico describe de manera detallada la arquitectura, componentes, funcionalidad y procedimientos de despliegue del software denominado API de Pronóstico Hidrológico Integrado (IDEAM). Este documento está destinado a servir como referencia técnica para el registro de la obra ante la Dirección Nacional de Derecho de Autor, así como para futuros desarrolladores, ingenieros de datos y administradores del sistema.

El software consiste en un microservicio de alta precisión desarrollado en Python bajo el framework FastAPI, especialmente diseñado para el pronóstico y alerta de caudales diarios utilizando arquitecturas avanzadas de ensamble de Gradient Boosting y modelos clásicos de series de tiempo. A lo largo de este manual, se detallan los módulos de código fuente, las tecnologías empleadas, la validación científica de los algoritmos y los pasos necesarios para su puesta en marcha en un entorno de producción contenerizado.





1. Objeto y finalidad del software

Este software se constituye como una API (Interfaz de Programación de Aplicaciones) de Backend, la cual actúa como el núcleo central para la lógica de negocio, la ejecución de modelos predictivos y la persistencia de datos relacionados con el monitoreo fluvial. Su finalidad primordial es ofrecer predicciones altamente fiables de caudales diarios para la estación hidrológica de referencia Pueblo Nuevo 2117100 (IDEAM), calibrada sobre una serie histórica de entrenamiento de largo plazo (2010–2017) con una ventana de validación fuera de muestra (hold-out) de 60 días.

Las funciones principales abarcan el procesamiento de series de datos diarias suministradas mediante solicitudes JSON estructuradas o cargas masivas de archivos de Excel, la validación automática de continuidad temporal, la extracción de características basadas en rezagos (lags) y estadísticas móviles, así como el cálculo en tiempo real del nivel de alerta hidrológica aplicable según el caudal proyectado.

Para efectos operacionales, el sistema evalúa el caudal medio proyectado para determinar los niveles de alerta según los siguientes criterios paramétricos:

Nivel	Condición	Descripción
LOW	$\text{media} < 1.5 \text{ m}^3/\text{s}$	Caudal bajo, seguimiento de rutina en la estación.
NORMAL	$1.5 \leq \text{media} < 8.0 \text{ m}^3/\text{s}$	Condiciones normales y estables de operación.
HIGH	$\text{media} \geq 8.0 \text{ m}^3/\text{s}$	Caudal elevado, activación de monitoreo intensivo.

- Enlace URL Github: https://github.com/OscarsI10/caudal_api.git





2. Arquitectura y Componentes Técnicos

El sistema implementa una arquitectura desacoplada y robusta por capas lógicas, optimizada para la ingesta intensiva de datos de series temporales y ejecución síncrona de pipelines de Machine Learning. El microservicio expone interfaces RESTful securizadas y mantiene una persistencia estructurada mediante el uso del framework FastAPI, Pydantic v2 para tipado y validación de esquemas, SQLAlchemy como ORM, y PostgreSQL 16 para el almacenamiento de datos indexados.

2.1. Estructura del Proyecto en Código Fuente

La distribución de los directorios y módulos fuentes estructurados dentro del repositorio del proyecto se detalla a continuación:

```
/api-pronostico-hidrologico
├── app/
│   ├── __init__.py          # Inicialización de la API FastAPI y CORSMiddleware
│   ├── main.py              # Configuración de sesión y motor de base de datos
│   ├── database.py
│   ├── models/
│   │   └── forecast.py      # Modelo relacional ORM para persistencia de pronósticos
│   ├── schemas/
│   │   └── forecast.py      # Contratos de validación de datos Pydantic v2
│   ├── routers/
│   │   └── forecast.py      # Definición de rutas y endpoints (/manual, /bulk)
│   ├── ml/
│   │   ├── engine.py        # Motor central de orquestación ML e inferencia
│   │   ├── pipeline.py       # Extracción de características temporales (Lag / Rolling)
│   │   ├── ensemble.py       # Arquitectura de los 4 modelos de pronóstico
│   │   └── classifier.py     # Clasificación de alertas según percentiles
│   └── services/
│       ├── charts.py         # Generación interna de gráficos Plotly HTML
│       └── forecast_service.py # Lógica transaccional e inyección de datos ORM
```



models_store/	# Artefactos PKL serializados (Volumen Docker montado)
ensemble_model.pkl	
scaler_features.pkl	
ensemble_feature_cols.pkl	
test_metrics.pkl	
serie_historica.pkl	
train_models.py	# Script independiente para entrenamiento y calibración
requirements.txt	# Dependencias mandatorias de librerías del entorno

2.2. Descripción de las Capas y Componentes

1. Capa de API y Enrutamiento (main.py, app/routers/): Constituye la interfaz de entrada del sistema. Controla el flujo de peticiones HTTP, gestiona las políticas de origen cruzado (CORS) y mapea las peticiones JSON o Multipart de carga masiva hacia sus respectivos controladores lógicos en el router de predicciones.

2. Capa de Validación y Esquemas (app/schemas/): Garantiza la calidad e integridad de los datos de entrada en tiempo de ejecución. Utiliza validaciones nativas de Pydantic v2 para comprobar de forma estricta los requerimientos operativos de las series temporales, rechazando de inmediato datos mal formateados o incompletos.

3. Motor de Machine Learning (app/ml/): Componente central enfocado en dos fases. En la fase fuera de línea, transforma las variables crudas mediante el cálculo de rezagos (1, 7, 14, 30 días) y ventanas móviles de rolling stats de 7 y 14 días con normalización MinMaxScaler. En la fase síncrona, consume el pipeline de ensamble de modelos serializados para retornar los caudales proyectados de forma óptima.

4. Capa de Servicios de Datos y Persistencia (app/services/, database.py): Separa la lógica de los endpoints de las llamadas a base de datos. Orquesta las transacciones ACID con el motor PostgreSQL a través de SQLAlchemy y se encarga de la generación en segundo plano de representaciones interactivas Plotly HTML consumidas por interfaces cliente externas.



2.3. Diagrama de Flujo de una Petición de Predicción

El siguiente diagrama ilustra cómo interactúan las capas al procesar una petición al endpoint POST /patients/{id}/predict:

[Cliente]

- |
- | 1. Petición POST con serie de datos (JSON o Excel .xlsx)
- ▼

[FastAPI / Uvicorn]

- |
- | 2. Enruta a @router.post en `routers/forecast.py`
- ▼

[Capa de Validación y Esquemas]

- |
- | 3. `schemas/forecast.py` intercepta la petición:
- | - Valida formatos de fecha ISO 8601 y tipos numéricos.
- | - Verifica la estructura mediante ForecastRequest.
- | - Retorna HTTP 422 si la estructura de datos es inválida.
- ▼

[Capa de Integridad Hidrológica]

- |
- | 4. `ml/pipeline.py` procesa y evalúa la serie temporal:
- | - Aplica alineación diaria obligatoria con `asfreq("D")`.
- | - Comprueba que existan mínimo 35 registros diarios continuos.
- | - Retorna HTTP 420 si se detectan nulos o fechas faltantes.
- ▼

[Capa de Machine Learning - Engine]

- |
- | 5. El código del motor predictivo se ejecuta en `ml/engine.py`:
- | - Extrae variables de rezago (Lags de 1, 7, 14 y 30 días).
- | - Calcula estadísticas móviles (Rolling stats de 7 y 14 días).
- | - Normaliza las características usando el MinMaxScaler cargado.
- | - Llama al método del modelo de Ensamble (Gradient Boosting .pkl).
- |



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

- | 6. Ejecuta la clasificación en ``ml/classifier.py``:
 - | - Evalúa la media proyectada contra los percentiles históricos.
 - | - Asigna el nivel de alerta correspondiente (LOW, NORMAL, HIGH).

V

[Capa de Servicios y Persistencia]

- |
 - | 7. ``services/forecast_service.py`` gestiona la salida y auditoría:
 - | - Registra el `forecast_id` (UUID v4) y metadatos en PostgreSQL.
 - | - Genera las visualizaciones interactivas HTML con Plotly.

V

[Respuesta]

- |
 - | 8. El resultado del pronóstico se empaqueta en un JSON.
 - | - ``schemas.ForecastResponse`` valida la salida de datos definitiva.

V

[Cliente]

3. Descripción de Módulos y Archivos Fuente

El núcleo funcional de la aplicación se encuentra encapsulado en módulos lógicos cohesivos que interactúan de forma secuencial al procesar solicitudes hidrológicas:

- **main.py:** Punto de arranque global. Instancia el objeto de aplicación FastAPI, define configuraciones de cabeceras CORS e integra de forma modular el enrutador de predicciones hidrológicas. Adicionalmente, expone el endpoint funcional de monitoreo de salud interna.
- **database.py:** Centraliza la conexión técnica relacional. Inicializa el motor de base de datos a través de una sesión declarativa e implementa el método de inyección de dependencia para asegurar el cierre de conexiones activas tras completar transacciones.
- **models/forecast.py:** Declara las tablas de base de datos como entidades nativas de Python mediante SQLAlchemy. Contiene las definiciones de columnas de la tabla forecasts que almacena marcas de tiempo, metadatos de series recibidas y predicciones de salida de caudal.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA

"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado, aplicando todos los requisitos de las normas ISO implementadas en sus sedes Neiva y Pitalito"



- **schemas/forecast.py:** Define las reglas de negocio estrictas para el consumo de la API mediante contratos de datos Pydantic v2. Implementa las siguientes restricciones operativas críticas de uso obligatorio para prevenir excepciones en el motor de machine learning:
 - **Mínimo de Registros:** Se exige un bloque obligatorio de mínimo 35 registros diarios continuos para el correcto cálculo de variables de rezago.
 - **Frecuencia Diaria Estricta:** Formato estricto de fechas ISO 8601 (YYYY-MM-DD) sin presencia de días faltantes ni valores nulos.
 - **Horizonte de Predicción:** Restringe el parámetro de proyección a un horizonte de tiempo máximo de 90 días.
- **routers/forecast.py:** Mapea los controladores HTTP expuestos. El método de procesamiento JSON valida las estructuras jerárquicas directas, mientras que el controlador masivo procesa solicitudes Multipart analizando y validando archivos Excel (.xlsx o .xls) con columnas mandatorias de Fecha y Caudal.
- **ml/pipeline.py y ml/engine.py:** Orquestan de forma secuencial la normalización matemática de los datos recibidos mediante MinMaxScaler, alinean la frecuencia diaria estricta (retornando un código de error HTTP 420 ante la detección de nulos o días faltantes) y cargan dinámicamente en memoria los artefactos binarios (.pkl) optimizados para inferencia inmediata.
- **train_models.py:** Script de administración ejecutable de forma independiente y autónoma. Se encarga de procesar las series del histórico de entrenamiento, calibrar hiperparámetros de los algoritmos y guardar de forma serializada los pipelines entrenados dentro del almacén persistente.

4. Guía de Despliegue y Puesta en Marcha

La infraestructura técnica del microservicio está diseñada para ser desplegada ágilmente de forma contenerizada, garantizando consistencia funcional entre entornos locales de desarrollo y entornos de producción.

Prerrequisitos:

1. Disponer de una cuenta en GitHub.



2. Motor de contenedores Docker Engine (Versión v24.0 o superior) y orquestador Docker Compose instalados de forma local en el servidor físico.
3. Disponibilidad de conexión a internet para la descarga inicial de imágenes base y librerías enumeradas en requirements.txt.

Despliegue de la Infraestructura de Datos:

El motor relacional PostgreSQL 16 debe ser configurado y levantado de forma aislada mediante un servicio containerizado en Docker expuesto internamente. Se debe asegurar el correcto montaje de volúmenes persistentes vinculados al directorio físico de la máquina host para salvaguardar el histórico de auditoría de las predicciones almacenadas y las configuraciones de migraciones de Alembic.

Calibración, Inicialización y Puesta en Marcha de la API

Antes de inicializar el servidor web de la API de producción por primera vez, es estrictamente obligatorio realizar la fase de entrenamiento offline de los modelos lógicos de machine learning para popular el directorio de almacenamiento. Siga la siguiente secuencia ordenada de comandos técnicos en la consola del sistema:

```
# Fase 1: Instalación de la totalidad de dependencias y frameworks requeridos en el entorno  
pip install -r requirements.txt
```

```
# Fase 2: Ejecución obligatoria del script de entrenamiento offline para generar y serializar los archivos PKL  
python train_models.py
```

```
# Fase 3: Aplicación secuencial de migraciones de base de datos relacional mediante Alembic  
alembic upgrade head
```

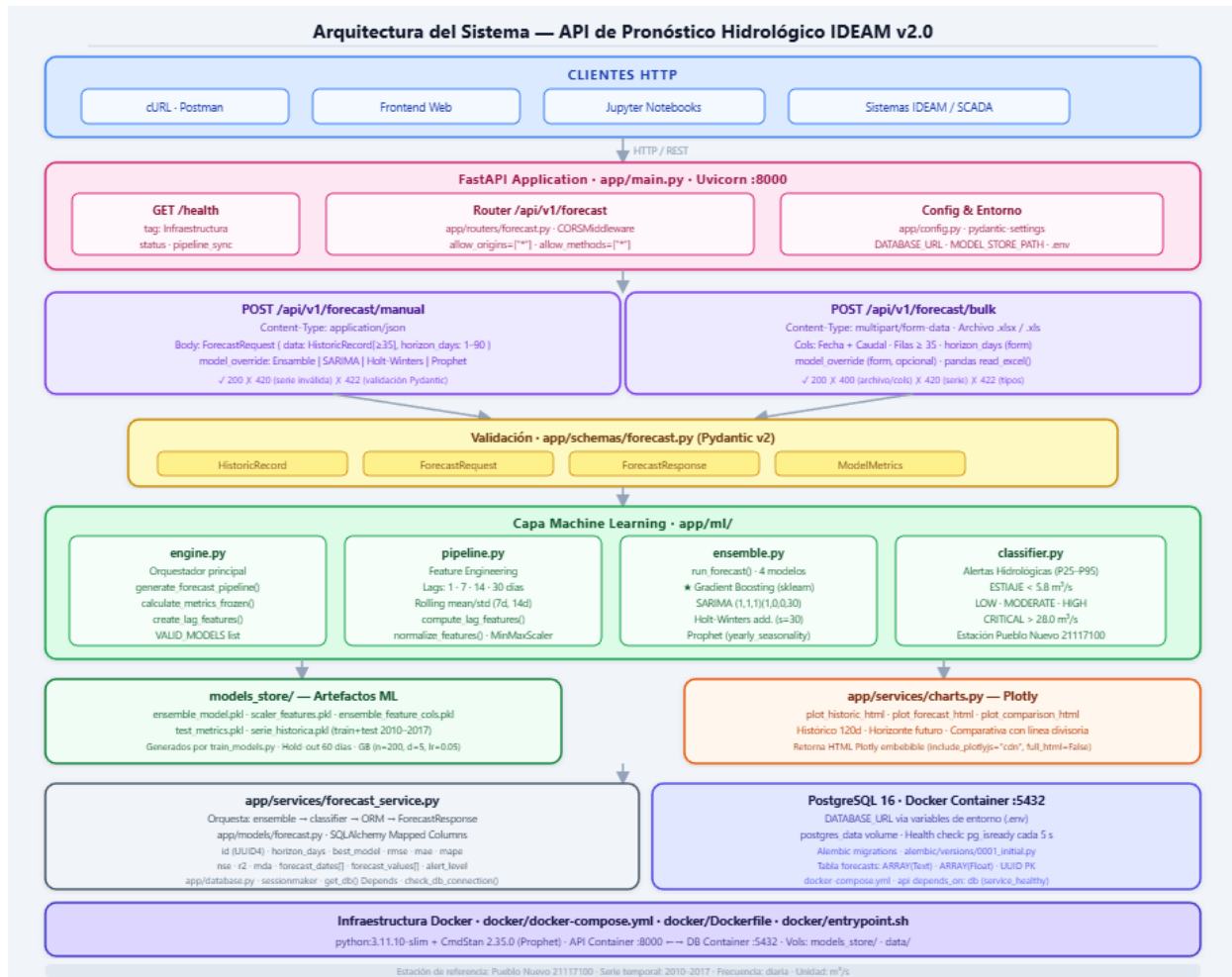
```
# Fase 4: Lanzamiento oficial del servidor web de producción a través del orquestador Uvicorn  
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Al completarse de manera exitosa el arranque del servicio, la terminal comenzará a registrar de forma continua las trazas logs de confirmación del microservicio, indicando el estado técnico saludable y la correcta sincronización del pipeline predictivo en memoria listo para la ingesta masiva de solicitudes hidrológicas.





Fig. 1 Diagrama de arquitectura por capas del microservicio.



Fuente: Autores

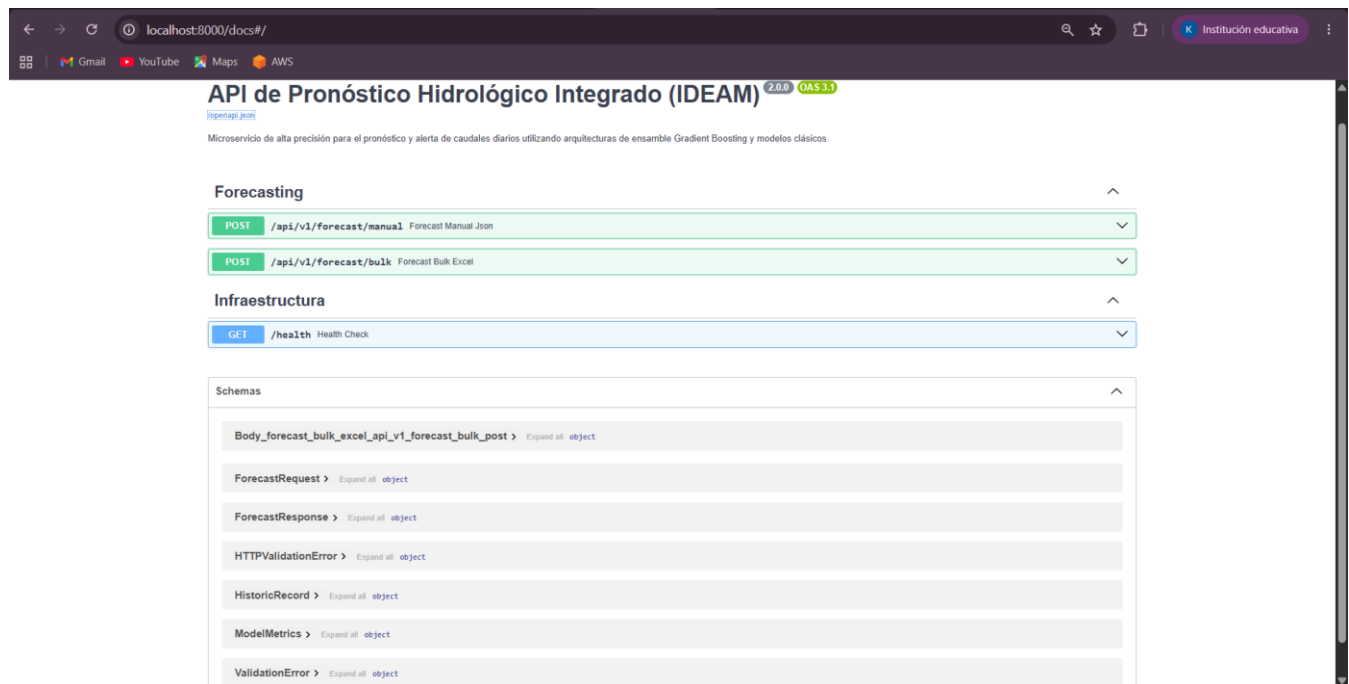


CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

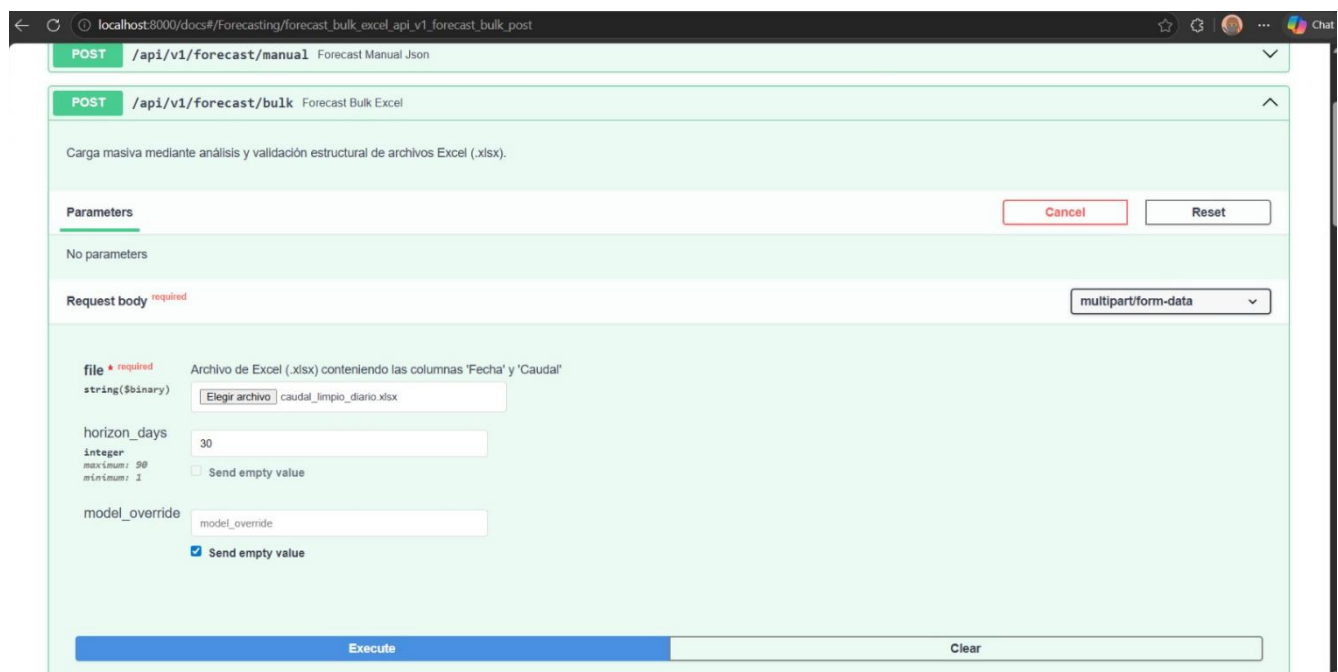
INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Fig. 2 Interfaz de Documentación de la API (Swagger UI)



Fuente: Autores

Fig. 3 Formulario de Carga Masiva de Datos en Swagger UI



Fuente: Autores

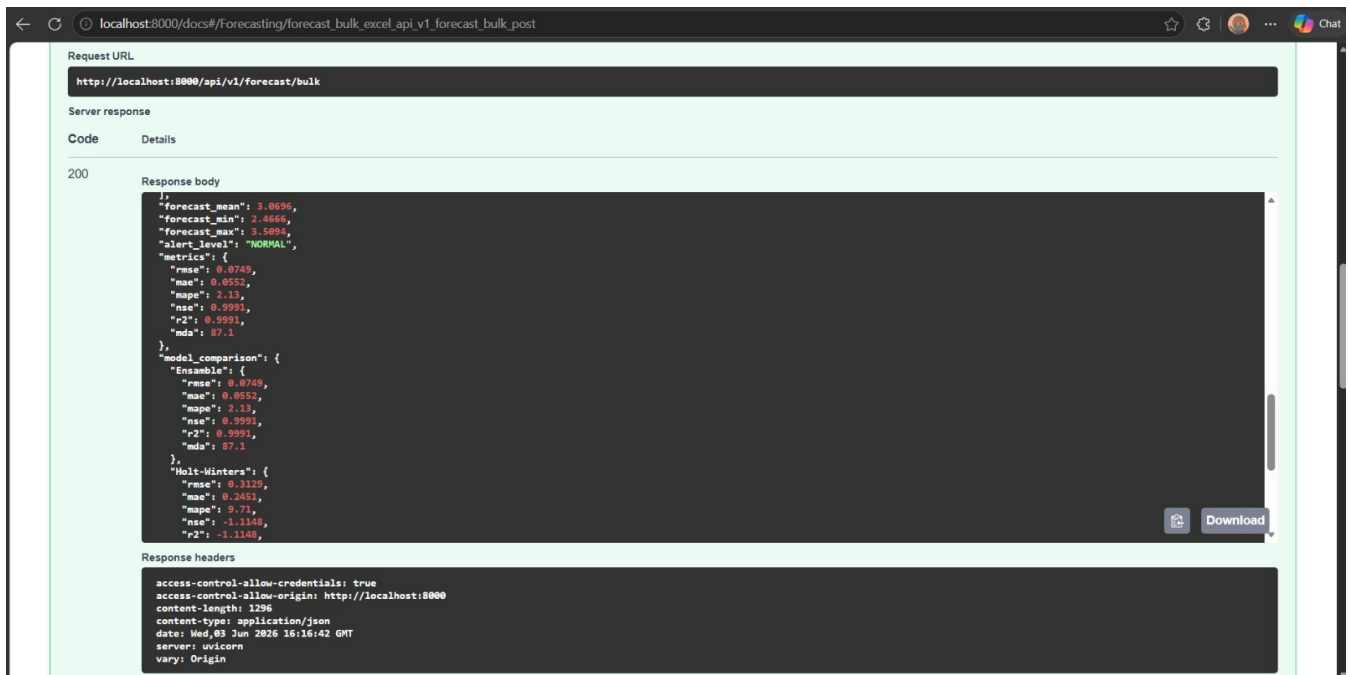
Sede Quirinal: Calle 21 No. 6 - 01
Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado, aplicando todos los requisitos de las normas ISO implementadas en sus sedes Neiva y Pitalito"



Fig. 4 Respuesta JSON Exitosa del Endpoint de Pronóstico



Fuente: Autores